



ITEC 204 – Data Structures and Algorithms

Name: Ken Patrick Layosa	Date: Septemeber 24 2026
Section: BSIT 2A	Instructor: Maxyne Nuela Ignacio

- I. **Objective:** Write a short explanation of what your program is designed to do. (Minimum of 3 sentences)

Answer:

- II. **Program Operations:** Briefly explain how your program performs each operation. (Insert Specific Code Line and Your Explanation)

Add Incident:

This program is designed to function as an IT Automation Incident Ticket Manager that helps a support team keep track of incident tickets generated by automation bots. It allows the user to add new incident tickets, display all currently active tickets, search for a specific ticket using its Incident ID, remove a ticket once it has been resolved, and display the total count of active tickets. The program applies the concepts of linear data structures by storing and manipulating all ticket records in a single Python list, and it uses a menu-driven interface so that a user can repeatedly choose which operation to perform without restarting the program.

Display Incidents: `incident_tickets.append(new_ticket)`

After the user types in the Incident ID, Bot name, and Short Description, the program first checks the list to make sure no existing ticket already has that same Incident ID. If the ID is unique, a new dictionary containing the three fields is created and added to the end of the `incident_tickets` list using the `append()` method, which is the standard way to insert a new record into a linear list structure.

Search Incident: `if ticket["Incident ID"].lower() == incident_id.lower():`
`print("Ticket found:")`
`...(print ticket details)`

The user is asked to type in the Incident ID they want to find. The program then performs a linear search it goes through the `incident_tickets` list one element at a time and compares each ticket's Incident ID to the one entered by the user. As soon as a match is found, the matching ticket's full details are displayed and the search stops; if the loop finishes without finding a match, the program reports that no ticket was found.

Remove Resolved Incident: `incident_tickets.remove(ticket)`

Similar to the search operation, the program performs a linear search through the list to locate the ticket whose Incident ID matches what the user typed in. Once the



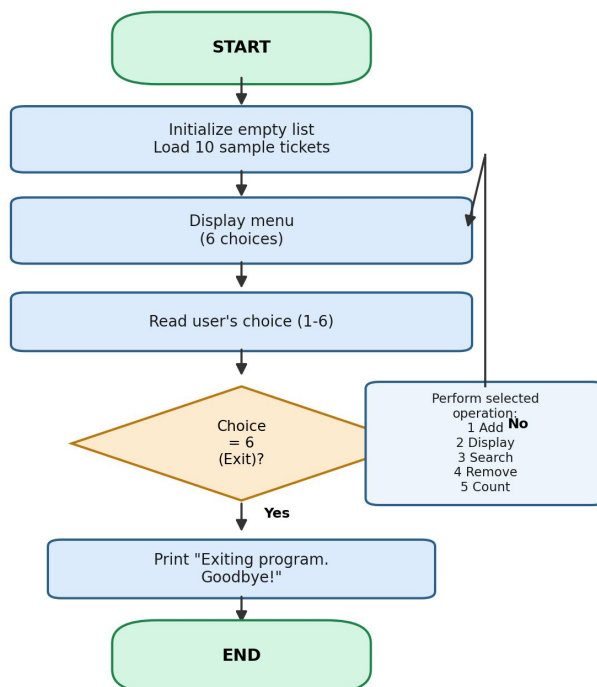
College of Computer Studies

matching ticket (dictionary) is found, it is deleted from the list using the remove() method, which represents the resolved ticket being taken out of the active incident list.

Count Active Incidents: `print(f'Total active tickets: {len(incident_tickets)}')`

The program uses Python's built-in len() function on the incident_tickets list, which returns the current number of elements (tickets) stored in it. This gives an instant count of how many incident tickets are still active without needing to manually loop through and tally them one by one.

III. **Flowchart:** Create a flowchart showing the overall process and logic of your program.



IV. **Conclusion:** Briefly explain what you learned from the activity and how the linear data structure was applied.

to model a real IT support workflow where records need to be added, viewed, searched, and removed in a straightforward, sequential manner. Because a list stores its elements in order and allows direct traversal from start to end, operations like displaying all tickets and counting active tickets



Republic of the Philippines
Laguna State Polytechnic University
Province of Laguna

College of Computer Studies

became simple to implement using loops and the built-in `len()` function, while `append()` and `remove()` made adding and deleting records efficient for this scale of data. I also learned the importance of linear search when working with lists, since finding or removing a specific ticket required comparing the Incident ID against each element until a match was found, reinforcing how the structure and its operations work together to organize real-world data.